

Parallel Ray Tracing

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

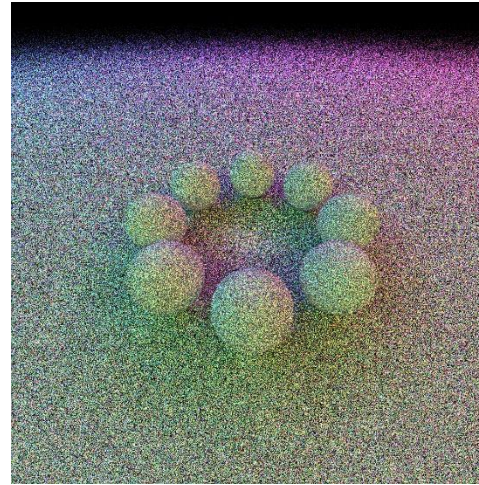
Cullen College of Engineering

University of Houston

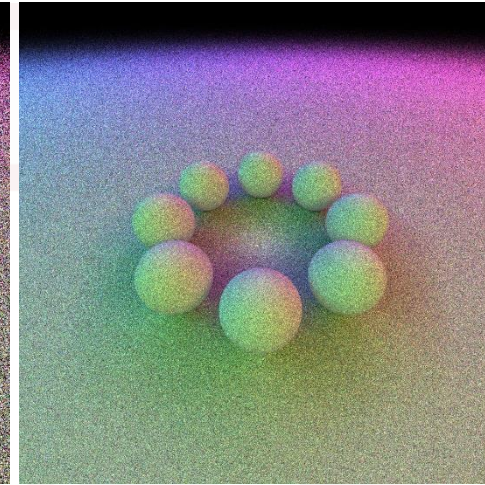
UNIVERSITY of **HOUSTON** | ECE

Programming Assignment 3

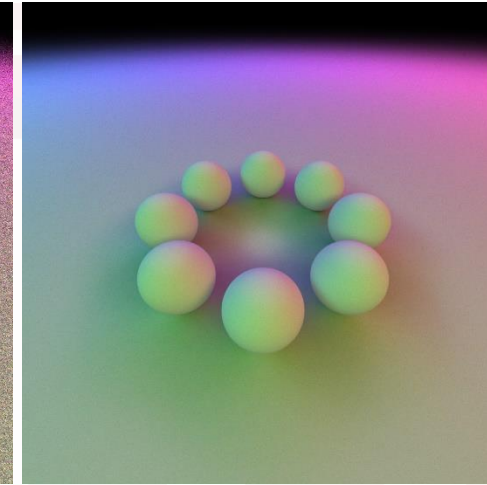
- Write a multi-threaded path tracer for spheres
- Some spheres can emit light (set emission color)
- Set a global maximum number of “bounces”
- Assign threads to improve performance
- Git repository as a zip file
- Repository contains a PDF report:
 - Speedup as a function of thread count
 - Flame graph
 - Sample images (with descriptions of the sphere layouts)



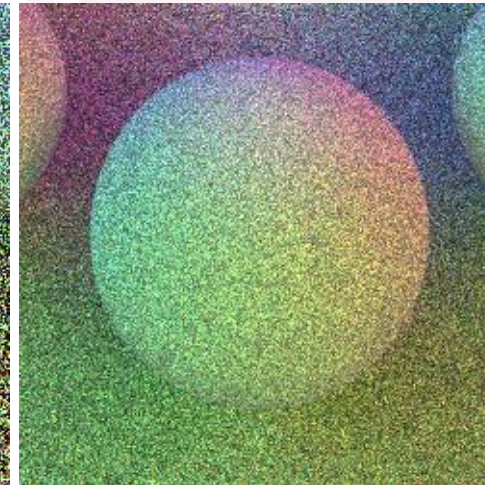
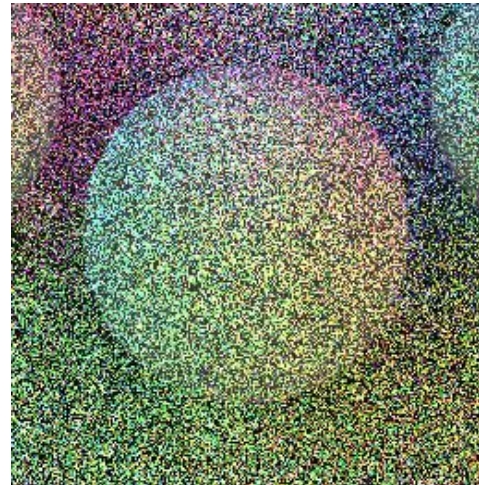
10 samples



100 samples



10000 samples



Implementation Tips

- Optimize single-threaded code first
- Write modular functions that can be callbacks
- Parameters: values that will be changed
- Globals: values that will be read-only
- CalcImage(...) example:
 - image pointer as parameter
 - image size as global
- You will ultimately add parameters that are unique to threads
 - ex. CalcImage(...) may eventually take a range of pixels to calculate

```
void CalcImage(...) {  
    // for all pixels  
    TracePath(r, ...);  
}  
void TracePath(...) {  
    // for all entities  
    // calculate intersections  
    // calculate next ray  
    TracePath(new_ray, ...);  
}
```

Implementation Tips

Single-Threaded Testing Using PA1

- Run your rendering function (ex. CalcImage) in the render loop:
- ```
while(!glfwWindowShouldClose(window))
{
 // ...
 CalcImage(...);
 // ...
}
```
- This will re-render the image every frame before displaying
  - Enables basic profiling and optimization

## Two-Threaded Testing using PA1

- Path tracing requires taking multiple samples per pixel – add a loop to the render function
- Specify a small number of “bounces” at first (maybe 2)
- Launch this function as a thread that runs in the background:

```
std::thread t(CalcImage, ...);
while(!glfwWindowShouldClose(window)) {
 // ...
}
```



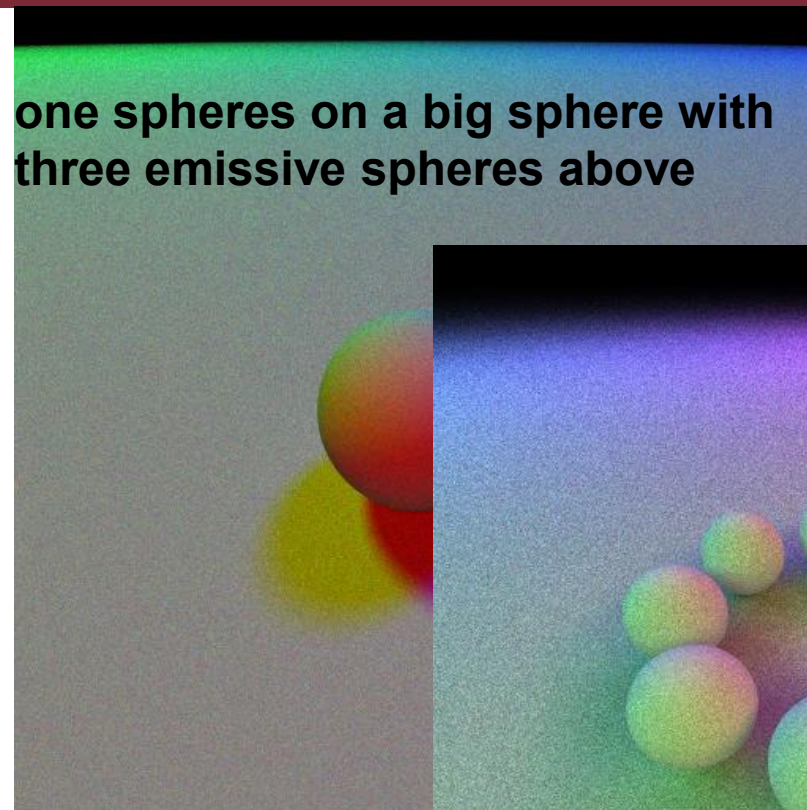
# Benchmarking

- Create some test scenes to profile your code
  - You can write these as hard-coded functions or design a simple file format
  - Keep it simple – you should be able to change obvious things with a few tweaks
  - (I wrote a python script to create a basic input file):

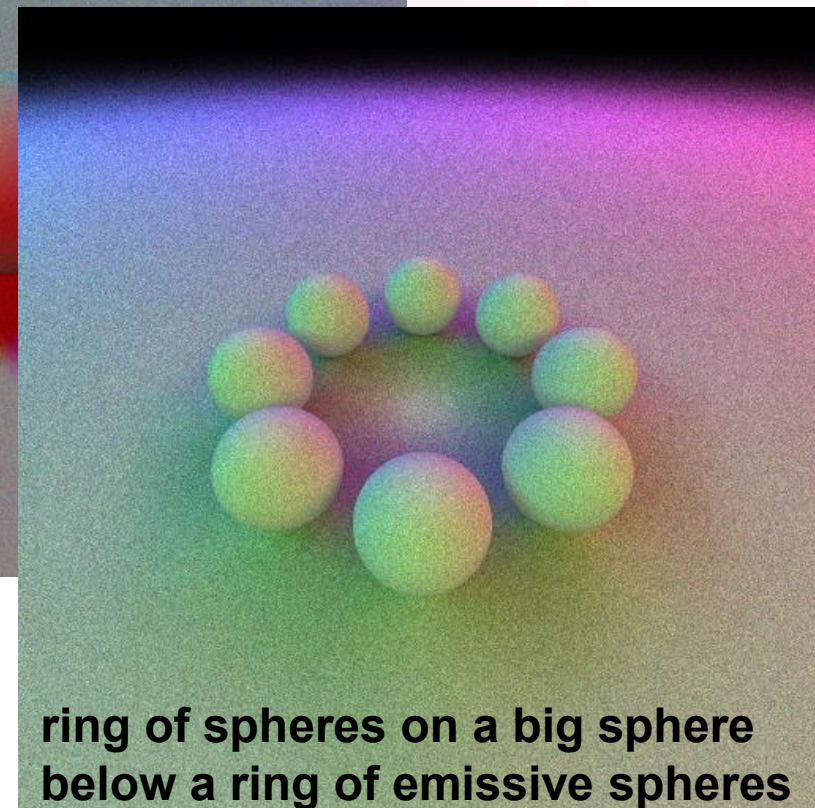
```
s 1 0 0 0 0 0 0 1 1 1 0
s 20 100 0 150 100 0 0 1 1 1 0
s 20 -70.7 -70.7 150 0 100 0 1 1 1 0
s 20 -70.7 70.7 150 0 0 100 1 1 1 0
s 9998.9 0 0 -10000 0 0 0 1 1 1 0
```



one spheres on a big sphere with  
three emissive spheres above

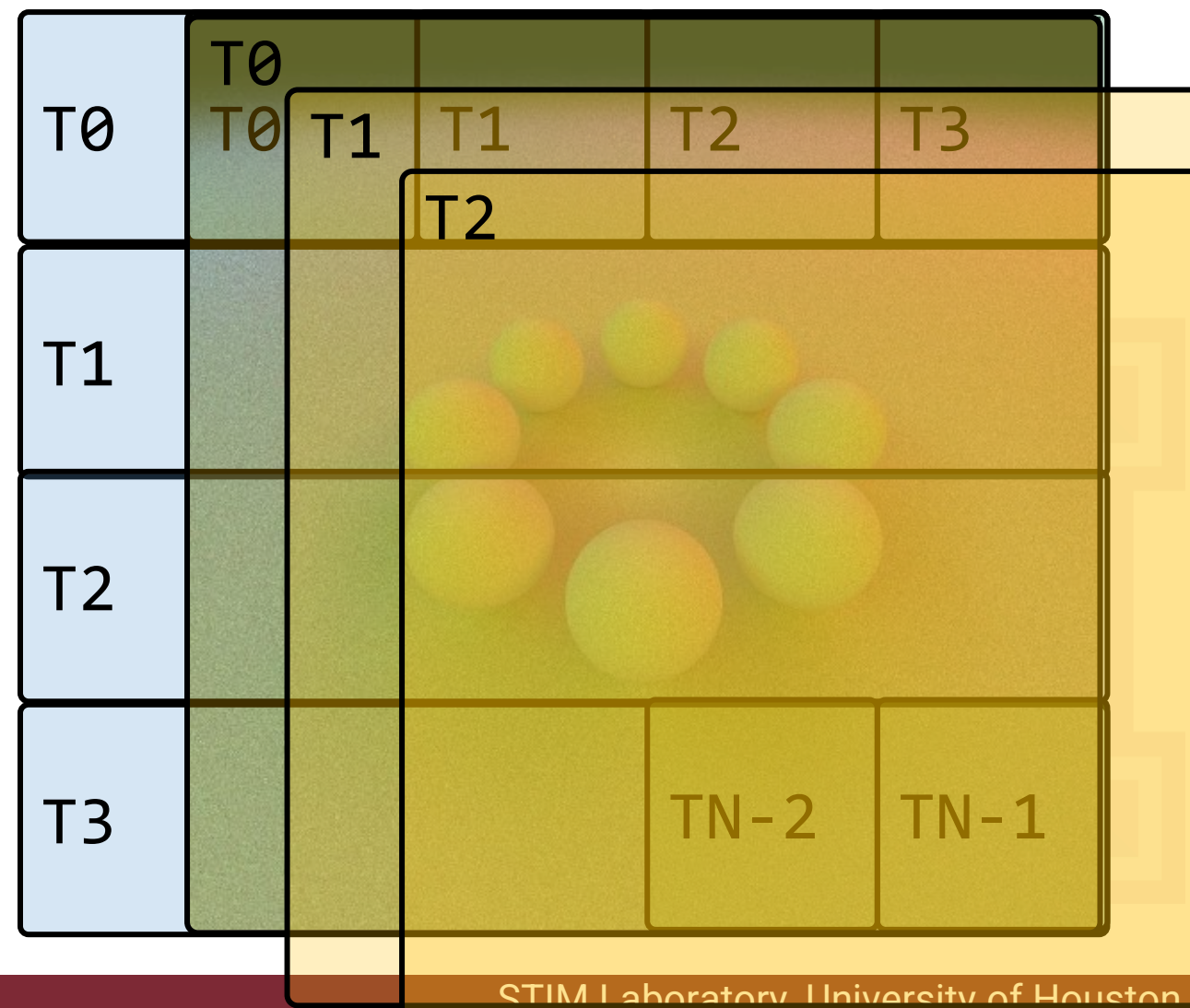


ring of spheres on a big sphere  
below a ring of emissive spheres



# Multiple Threads

- How should you parallelize this process?
- Blocking along one axis
  - Submit a range of pixels to the render function
- Blocking along both axes
  - Submit two ranges of pixels to the render function
- Threads over samples
  - Calculate separate images and average them

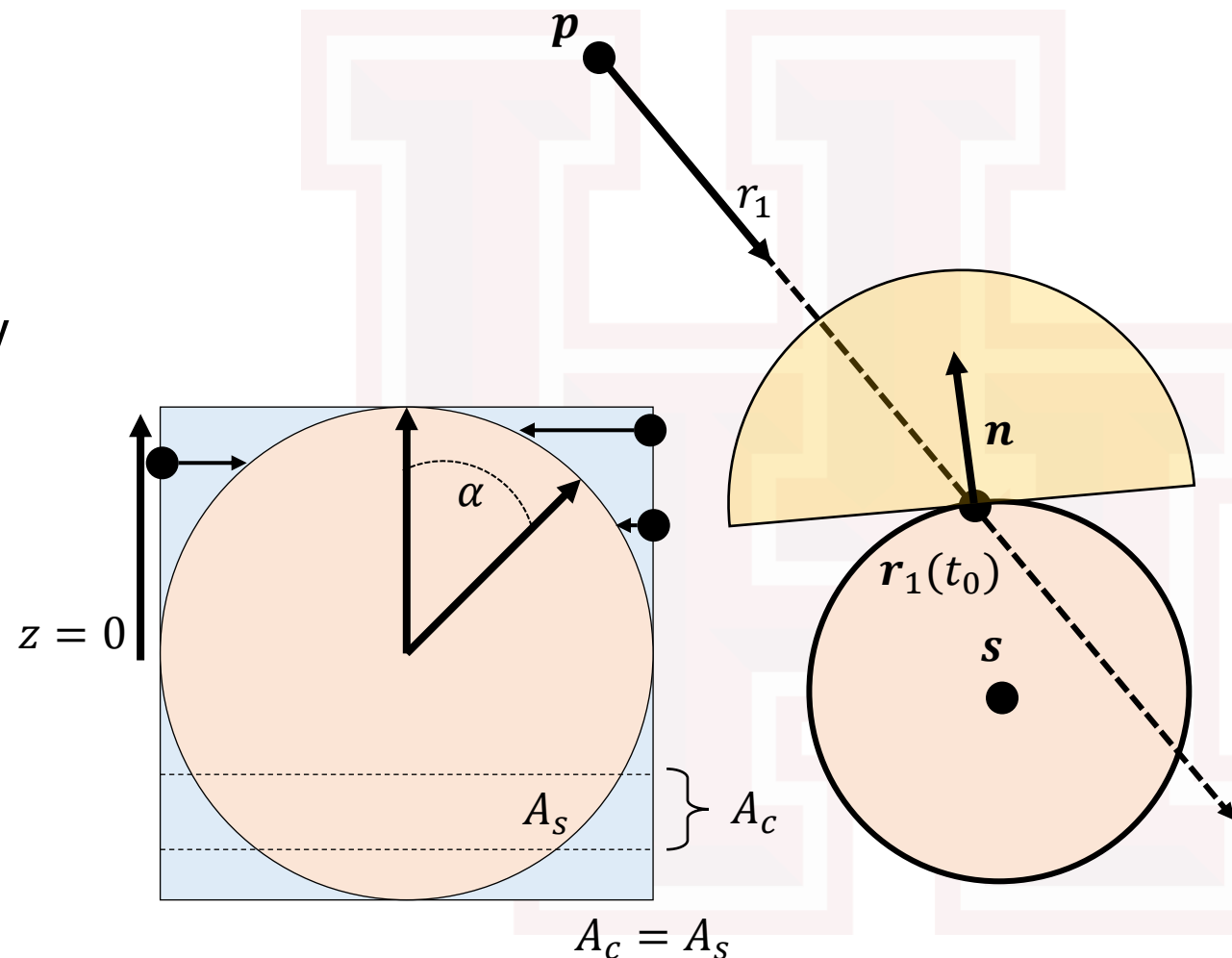


# Implementation Math

- Calculate the next path ray
  1. Calculate the normal  $n$
  2. Sample a hemisphere around  $n$
- Archimede's theorem:
  - The area of a sphere equals the area of every right circular cylinder circumscribed about the sphere
- Sampling an angle  $\alpha$  about  $z$  given two random numbers:

$$z_r \in [\cos \alpha, 1] \quad \theta_r \in [0, 2\pi)$$

$$\mathbf{v} = \begin{bmatrix} (1 - z^2) \cos \theta \\ (1 - z^2) \sin \theta \\ z \end{bmatrix}$$



# Debugging Tips

Highly recommend using ImGui

- Add UI elements that can change parameters during rendering
- Output timings directly to observe changes
- Changing parameters can help you in several ways:
  1. Makes your code more robust – forces you to deal with memory leaks
  2. Change camera angles relative to spheres can help find math errors

